

Playing Cards Detection

과목명	송현주 교수님
교수명	컴퓨터비전
학 과	컴퓨터학부
학 번	20233150
이 름	이경민
제출일	2025.11.28

목차

1. 알고리즘 개요	3
1.1 핵심 아이디어	3
2. 검출 및 분류 아이디어	5
2.1 Case1	5
2.2 Case2	8
3. 한계/오류 사례 및 개선 사안	18

1. 알고리즘 개요

1.1 핵심 아이디어

검출 단계에 들어가기 앞서서 입력 이미지에 겹친 카드의 유무를 따져서 겹친 카드가 있으면 겹친 카드 문제를 해결하는 로직을 실행시켰고 겹친 카드가 없다고 판단되면 겹친 카드가 없는 문제를 해결하는 로직을 실행시켰습니다.

겹친 케이스(Case 2)를 해결하는 로직 하나만으로도 겹치지 않는 케이스(Case 1)도 해결이 가능한 하지만 겹치지 않는 경우는 겹치지 않는 문제를 해결하는 로직이 더 확실하게 해결해주기 때문에 로직을 2개로 분류시켰습니다.

[케이스 분류 로직]

케이스는 크게 2가지로 분류했습니다.

- 겹침 없음(non_overlap) → 코너 기반 카드 검출 → 평탄화 → 분류 파이프라인
- 겹침있음(overlap) → 겹친 카드 전용 라벨링/분할 파이프라인

제일 먼저 Otsu 이진화를 시도해서 카드와 배경을 분리했습니다. 이미지에 조명 영향을 받는 부분 근처에 있는 카드는 Otsu를 통해서 검출이 안 되므로 이 문제를 해결하기 위해 Otsu를 먼저 수행하고 그 결과에서 큰 물체의 컨투어를 분석했습니다.

컨투어 분석을 통해서 화면 테두리에 붙어있는 큰 blob이 있어 조명/노이즈로 판단되는지 여부, 겹침/합쳐진 카드로 의심되는 물체가 하나라도 있는지 판단했습니다. 각각의 카드가 겹치지는 않았는데 아주 가까이 붙어있는 경우 큰 덩어리 하나로 판단했기에 여유 있게 0.58 ~ 0.82 사이여야 "단일 카드"로 인정하는 로직을 추가했습니다.

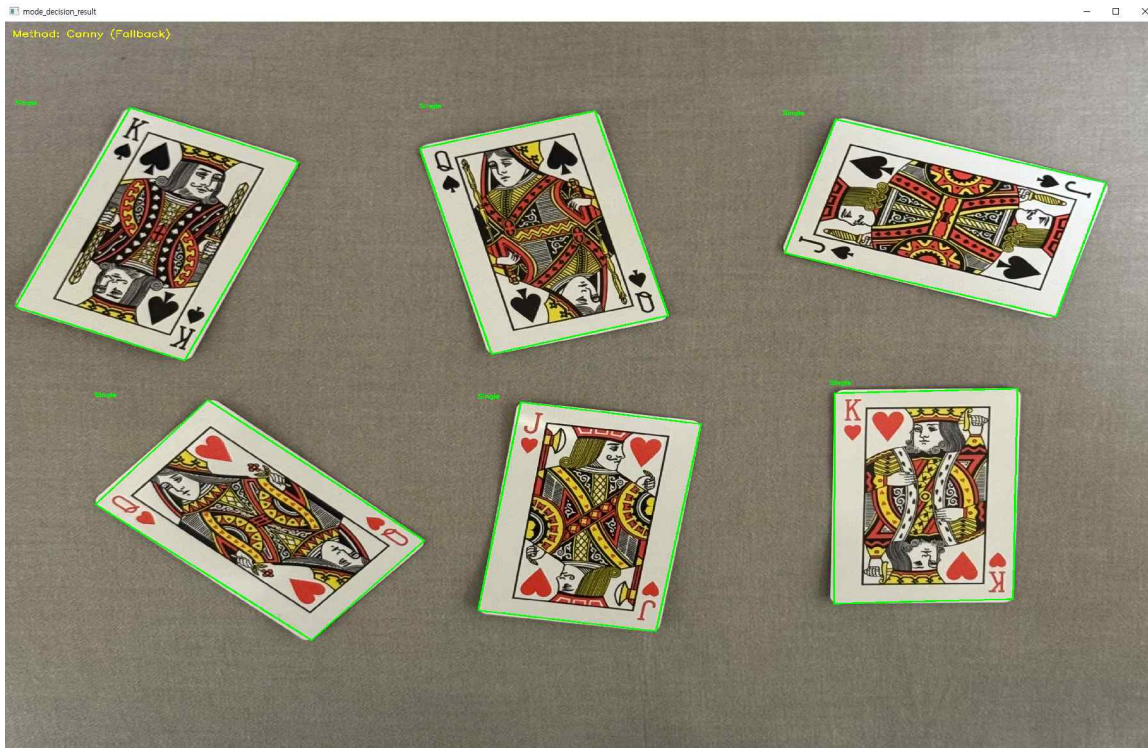


육안으로 멀쩡해 보이지만 Otsu 결과를 보면 우상단쪽에 조명 영향을 받는 카드가 보입니다.



이런 경우에는 적응형 이진화 과정을 통해 각 카드를 배경과 분리했습니다.





2. 검출 및 분류 아이디어

2.1 [Case1(겹치지 않은 케이스) 검출 및 분류]

카드 검출 이후 각 카드의 좌상단에 있는 rank/suit 정보를 얻기 위해 좌상단 코너를 동일한 크기를 잘라낸 이후 미리 준비해둔 템플릿 이미지 이용해서 템플릿 매칭을 시도했습니다.

하지만 이미지에 카드가 다 똑바로 서있지 않는 경우도 있기 때문에 카드들을 다 똑바로 세워주는 로직이 필요했습니다.

우선 평탄화 작업을 하기 위해서 `find_flatten_cards` 함수를 사용했습니다. 이 함수는 각 카드마다 4개의 코너 좌표가 모여있는 리스트를 전달받아서 `getPerspectiveTransform`으로 4점 homography를 구했습니다. 이 행렬 matrix를 이용하면 기울어져 있거나 원근감이 들어가 있거나 약간 찌그러져 있는 카드 영역을 정사각형 격자 위의 직사각형으로 매핑할 수 있습니다.

카드가 사진 속에서 눕거나 회전되어 있을 수 있기 때문에 코너 순서와 가로/세로 방향을 항상 같은 기준으로 맞추는 과정이 필요했습니다. 카드 후보 박스를 만들기 위해서 `_order_box_points` 함수를 사용했고 최종 코너 결과를 한번 더 정리할 때 `correct_card_orientation` 함수를 사용했습니다.

`_order_box_points`는 먼저 기하학적으로

$(x + y)$ 가 최소인 점 $\rightarrow$ Top-Left

$(x + y)$ 가 최대인 점 $\rightarrow$ Bottom-Right

$(x - y)$ 가 최소인 점 $\rightarrow$ Bottom-Left

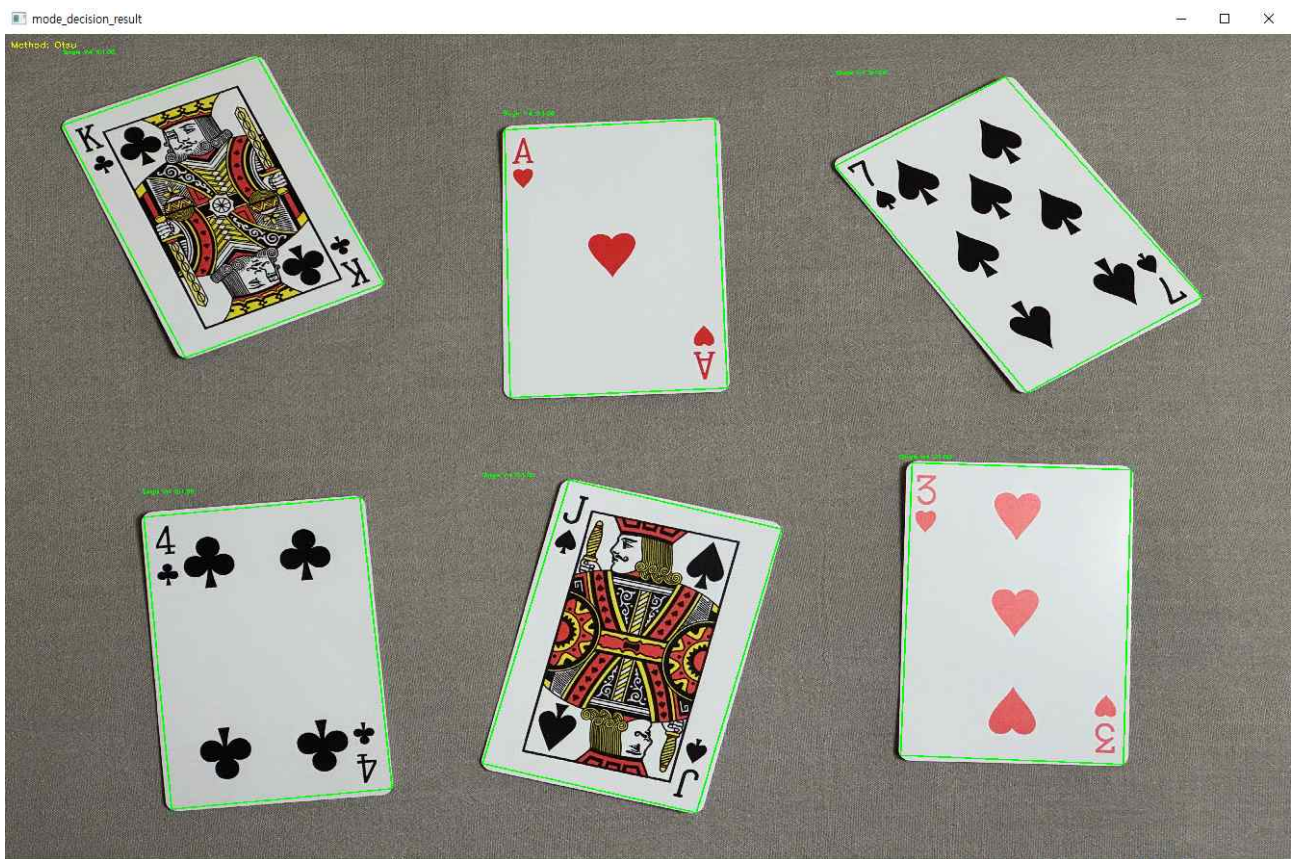
$(x - y)$ 가 최대인 점 $\rightarrow$ Top-Right

를 찾아서 [TL, BL, BR, TR] 순으로 정렬합니다.

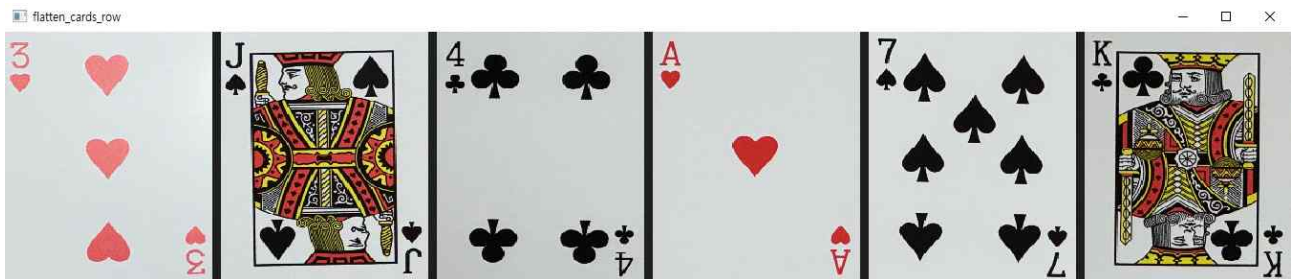
그 다음

TL→BL 거리 = 세로 길이(h), TL→TR 거리 = 가로 길이(w)로 계산합니다. 만약 $w > h$ 라면, “카드가 가로로 누워 있다”고 판단하고 [TL, BL, BR, TR] → [BL, BR, TR, TL] 순으로 한 칸씩 회전시킵니다. 즉, 좌표를 90도 돌려서 항상 세로가 더 긴 카드로 맞추고 이 단계 덕분에, 검출된 박스에서 나오는 코너들이 이미 “세워진 카드” 기준으로 정렬됩니다.

correct_card_orientation 함수는 최종적으로 얻어진 코너들에 대해 한 번 더 세로/가로를 확인해서, 여전히 가로가 더 길면 다시 90도 돌려줍니다.



이렇게 심하게 회전된 카드가 있어도 이 로직을 통해서 똑바로 세워집니다.

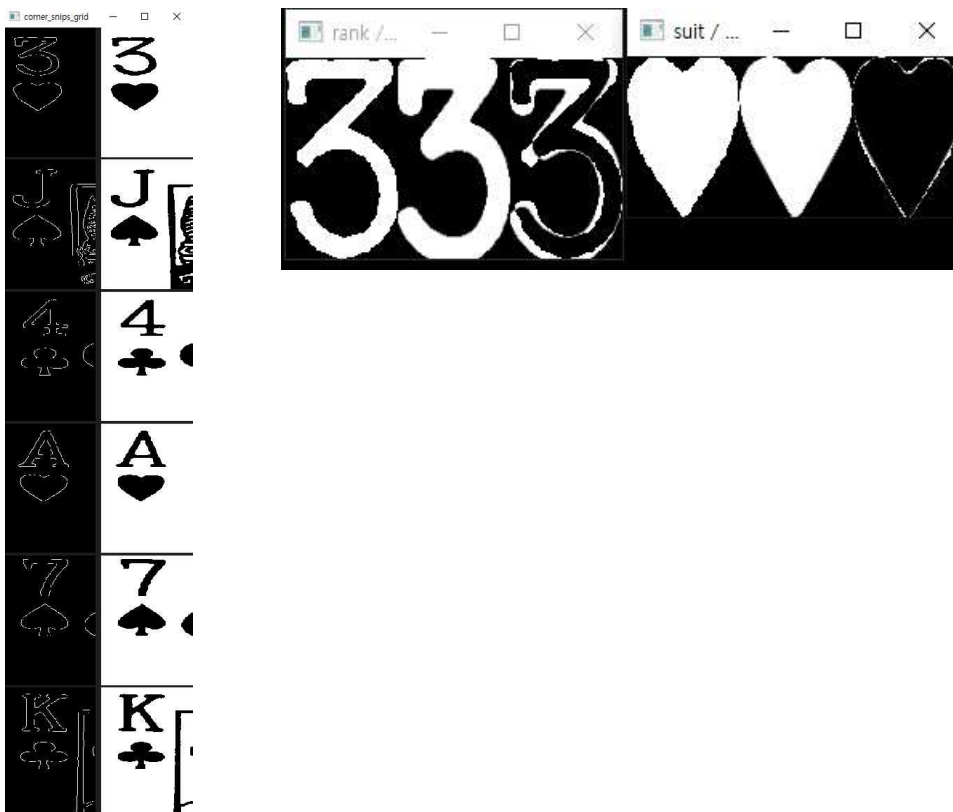


이렇게 똑바로 카드를 세운 이후 각 카드의 좌상단만 잘라서 각각을 템플릿 매칭을 해서 분류했습니다.

템플릿 매칭 `template_matching` 함수를 사용했습니다. 인자로 `rank`, `suit`, `train_ranks`, `train_suits`로 전달 받습니다.

- `rank`: 현재 카드에서 잘라낸 숫자/문자 영역
- `suit`: 현재 카드에서 잘라낸 무늬(♠♥♣♦) 영역
- `train_ranks`: 템플릿으로 사용할 `rank` 이미지들 리스트
각 원소는 `train_rank.img` (이미지) + `train_rank.name` (예: "A", "2", "K" ...)
- `train_suits`: 템플릿으로 사용할 `suit` 이미지들 리스트
각 원소는 `train_suit.img` + `train_suit.name` (예: "S", "H", "D", "C")

템플릿 매칭의 핵심 아이디어는 이진화된 입력 `rank/suit`와 템플릿 이미지들 간의 픽셀 차이(절댓값 합)을 구해서, 가장 차이가 작은 템플릿을 선택하는 방식입니다.



`cv2.absdiff` + 픽셀 합

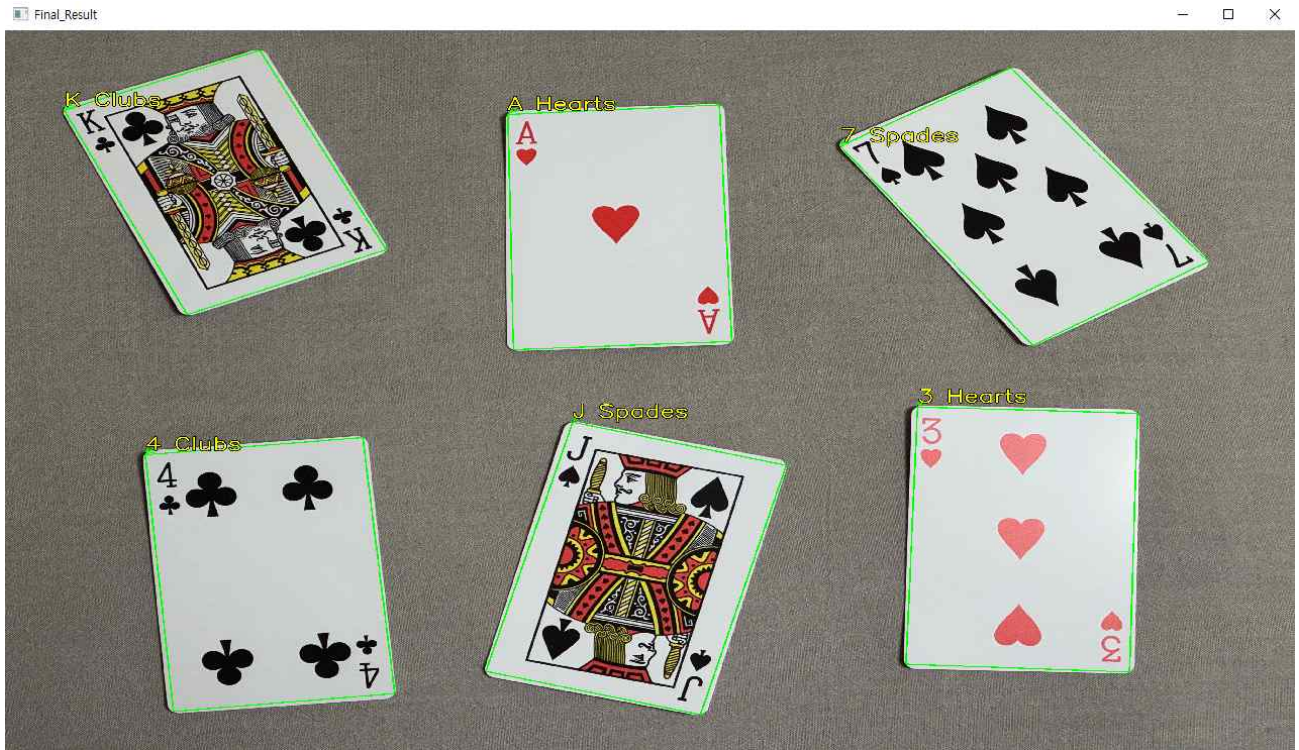
```
diff_img = cv2.absdiff(rank, train_rank.img)
```

- 두 이미지를 같은 위치 픽셀끼리 뺀 뒤 절댓값을 취한 이미지입니다.
- `rank/suit`는 이미 이진화 + 반전 상태라서 (배경=0, 글자=255)
동일한 픽셀은 0, 다른 픽셀은 255 정도의 값이 됩니다.
- `rank_diff = int(np.sum(diff_img) / 255)`
`diff_img` 전체 픽셀을 더한 다음 255로 나누고 결과적으로 “서로 다른 픽셀 개수”에 비례하는 스코어가 됩니다. 값이 작을수록 템플릿과 더 비슷한 `rank`라는 뜻입니다.

최적 rank 템플릿 선택

처음엔 best_rank_match_diff를 큰 값(10000)으로 초기화합니다. 각 train_rank에 대해 rank_diff를 계산하고 $\text{rank_diff} < \text{best_rank_match_diff}$ 이면 현재 템플릿을 새로운 best로 갱신합니다. 그때의 train_rank.name을 best_rank_match_name에 저장하고 diff_img도 디버깅용으로 저장합니다.

결국, 모든 rank 템플릿에 대해 픽셀 차이 합을 계산한 뒤 가장 작은 스코어를 가진 템플릿의 이름이 최종 rank 후보가 됩니다.



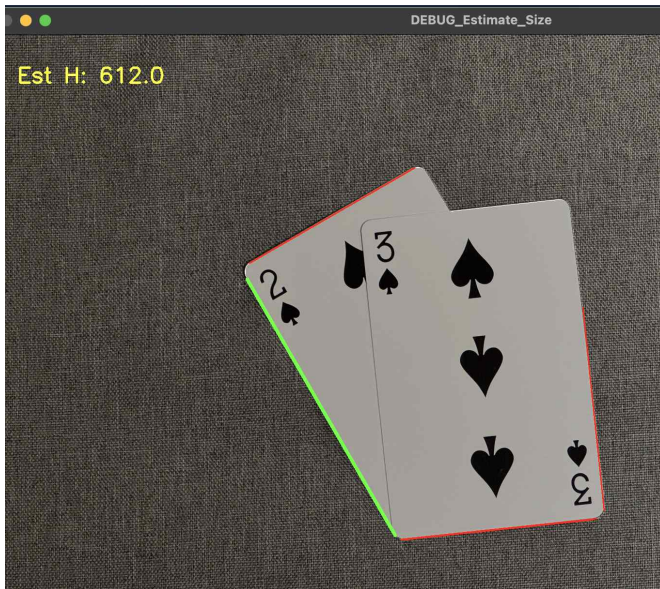
suit 템플릿 매칭 로직도 비슷한 패턴입니다.

2.2 [Case2(겹친 케이스) 검출 및 분류]

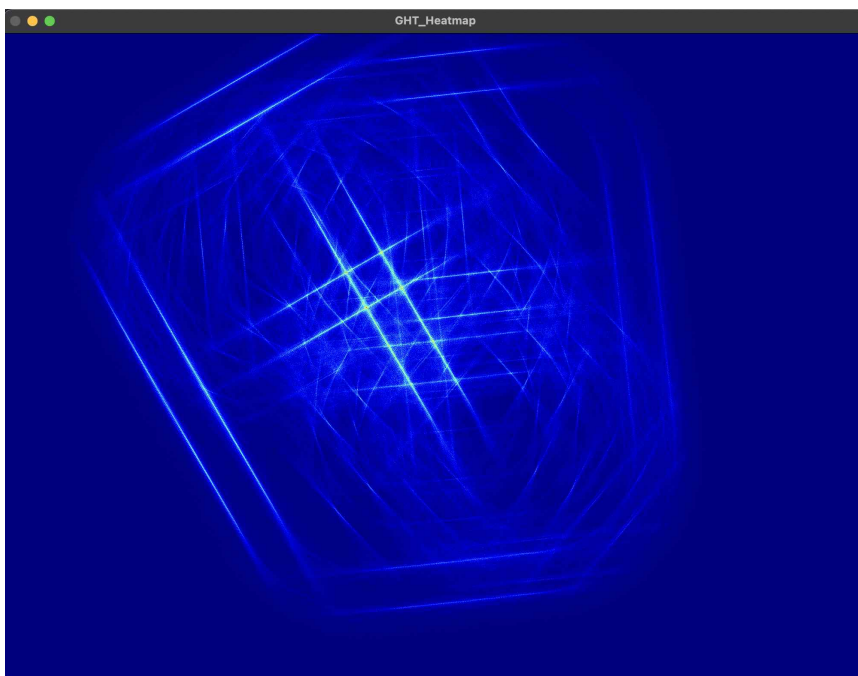
1. GHT 방식 시도

처음에 겹친 케이스 문제를 해결하기 위해 인터넷 검색을 하다가 이 문제를 해결한 논문을 참고했습니다. Generalized Hough Transform 기반의 사각형 투표 방식을 사용해서 카드 외곽의 일부만 보여도 많은 contour 점들이 사각형 후보에 표를 던져서 표가 가장 많이 모인 사각형이 실제 카드일 확률이 높다고 판단했습니다.

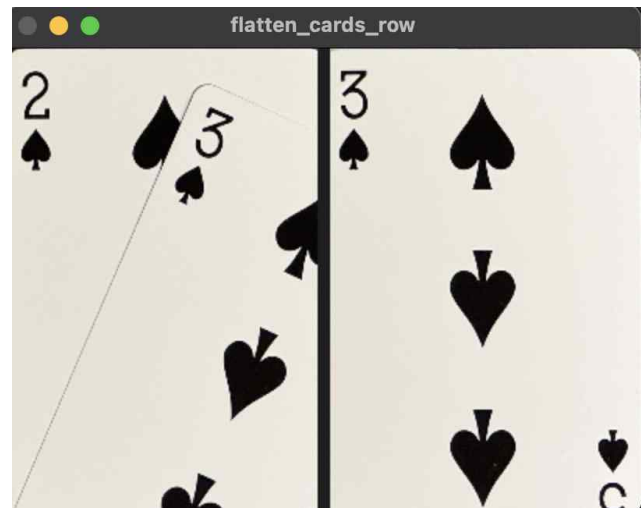
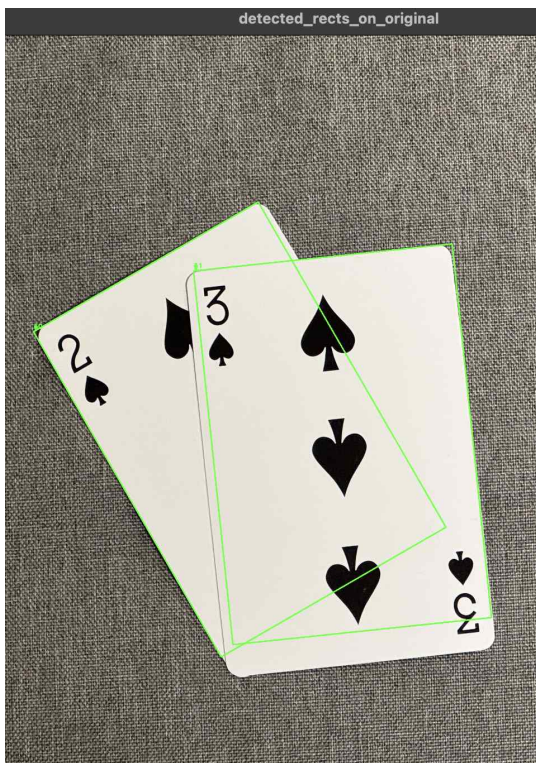
이 과정을 통해서 카드의 중심점을 유추하고 그 중심점을 기준으로 카드 영역을 유추한 다음에 Case1에서 사용한 분류 로직을 사용했습니다. 하지만 이 방식은 이미지에서 카드의 크기를 모르면 제대로 동작하지 않는 한계가 있었습니다. 이 문제를 해결하기 위해서 허프 직선 검출을 통해서 카드의 긴 변 위치를 찾고 비율을 이용해서 가로 세로 길이를 구하고 투표 방식을 통해 카드 중심점을 찾았습니다.



허프 직선 검출을 통해서 카드의 세로 길이를 찾을 수 있었습니다.



히트맵을 시각화하여 어디에 많이 투표됐는지 확인을 할 수 있었습니다.

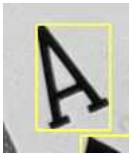


이렇게 중심을 찾으면 각 카드의 외곽 유추가 가능해지고 그 이후는 Case 1 로직을 사용해서 분류를 진행했습니다.

2. labeling 방식 사용

GHT 방식은 로직이 매우 복잡했고 완벽하기 구현하는데 어려움이 있었습니다. 그결과 카드가 여러장 많이 겹쳐있거나 특이한 형태로 겹쳐있는 경우에 카드의 크기를 추정하는 과정, 카드의 중심점을 찾아내는 과정에서 문제가 생겼습니다. 이러한 문제점을 해결하기 위해서 labeling 방식을 도입했습니다.

라벨링 방식만으로 검출된 것을 템플릿 매칭에 사용하기에는 큰 문제점이 있었습니다. 만약에 회전된 카드의 rank/suit에 라벨링 박스가 생겼을 때 회전 문제 때문에 템플릿 매칭이 잘 되지 않는 문제가 발생했습니다.



이 문제를 해결하기 위해서 라벨링된 박스 영역을 ROI로 설정하고 해당 ROI에서 객체의 컨투어를 구한 다음에 minAreaRect가 주는 회전된 사각형 정보를 이용해서 이미지에서 해당 부분만 똑바로 세워주는 시도를 했습니다. 객체를 조금씩 회전해가면서 높이가 제일 크게 나온 순간이 똑바로 섰다는 가정을 사용해서 rank들이 똑바로 세웠습니다.

그 이후 Case1에서 사용한 템플릿 매칭을 하기 위해 rank와 suit를 하나로 병합해주는 시도를 했습니다.

가장 먼저, 검출된 모든 박스를 대상으로 이중 for 문을 돌면서 현재 박스와 가장 적합한 짝을 찾습니다. 단순히 거리만 가까운 것이 아니라, 이 두 개가 정말 하나의 카드 코너에 있는 숫자와 문양인가?를 판단하기 위해 4단계의 필터를 거칩니다.

A. 1차 거리 필터

두 객체 사이의 거리가 객체 크기의 합보다 3배 이상 멀리 떨어져 있다면, 아예 짝이 아니라고 판단하고 계산을 생략합니다.

B. 수직 정렬 확인

카드가 기울어져 있을 때, 단순히 x, y 좌표만 비교해서는 위아래로 정렬된 것인지 알 수 없습니다. 이 코드는 벡터 내적을 사용하여 회전된 축을 기준으로 정렬 상태를 확인합니다.

- 기준 축 설정: current 박스의 긴 변을 찾아 Y축(세로 방향)으로 가정하고, 이에 수직인 벡터를 X축(가로 방향)으로 정의합니다.
- 상대 위치 투영: 두 박스의 중심점을 잇는 벡터를 구한 뒤, 이를 current 박스의 가로 방향 단위 벡터와 내적합니다.
- 판단: 이 lateral_offset 값이 작을수록 두 객체는 같은 세로축 선상에 있는 것입니다. 코드는

이 오차가 객체 너비의 1.1배~1.5배 이내일 때만 수직 정렬로 인정합니다.

C. 종횡비 검사

일반적으로 카드의 rank는 세로로 긴 형태이고, suit은 둥글거나 넓데데한 형태입니다. 만약 두 객체가 모두 뚱뚱하다면, 이는 숫자 없이 문양만 2개 잡혔거나 노이즈일 확률이 높으므로 병합하지 않습니다.

D. 노이즈 제거

종횡비가 5.0을 넘어가면 아주 가느다란 선(카드의 테두리나 찢어진 자국 등)일 확률이 높습니다. 이런 노이즈가 문양과 합쳐지는 것을 방지합니다.

E. 2차 거리 필터

위의 기하학적 조건을 모두 통과했더라도, 최종적으로 거리가 너무 멀면(1.4배 초과) 짝이 아니라고 봅니다. (예: 다른 코너에 있는 문양끼리 묶이는 것 방지)

3. 최종 병합

위 조건들을 통과하여 가장 적합한 짝(pair_idx)을 찾았다면, 두 박스를 하나로 합칩니다.



하지만 이런 철저한 검사를 거쳤음에도 이미지의 크기나 이미지 안에서의 카드의 특성에 따라

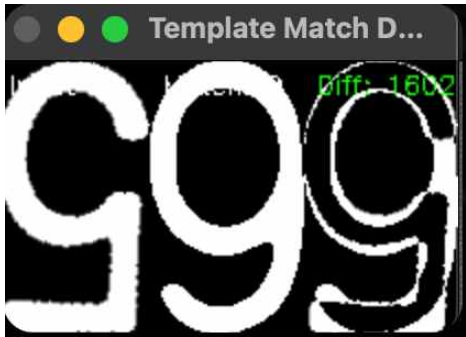
근처에 있는 rank/suit가 아닌 엉뚱하게 묶이거나 카드가 간혹 너무 가까이 붙어있으면 바로 옆에 겹쳐있는 rank끼리 묶이는 경우가 발생했습니다.

각종 시도를 해보다가 그냥 묶은 다음에 템플릿 매칭을 하지 않고 각각을 템플릿 매칭을 시도하는 새로운 로직을 도입했습니다.

라벨링으로 찾은 각각의 객체를 ROI로 잘라서 기울기를 보정하는 것은 같았으나 매칭 시도시 해당 객체를 회전해가면서 템플릿 이미지와 비교하는 작업을 진행했습니다.

match_with_rotation_search 함수에서 이진 이미지에 대해 정방향(0도 주변) 검색을 하고 180도 뒤집어서 한 번 더 체크해서 두 결과 중 diff(픽셀 차이)가 더 작은 쪽을 최종 후보로 채택했습니다.

180도로 뒤집어서 한번 더 체크했던 이유는 라벨링 방식을 사용하면 Case1처럼 좌상단만 볼 수 있는 것이 아닌 카드 전체 영역을 보기 때문에 카드 우하단에 위치한 rank/suit도 같이 검출이 됐도 180도로 뒤집지 않으면 엉뚱한 템플릿 이미지에 매칭되는 문제가 있었습니다.



(5 스페이드 카드 우하단에 있는 rank 5를 9에 매칭시키는 문제가 발생했었음)

180도 확 뒤집어진 경우가 아닌 조금씩 회전된 경우도 있었기에 세밀한 회전 검색 과정도 같이 사용했습니다.

각 ROI에 대해 -45도에서 +45도까지 5도 단위로 회전하면서 crop_content로 글자 부분만 다시 잘라내고 rank 템플릿과 비교, suit 템플릿과 비교했습니다.

absdiff → np.sum(...) / 255 로 서로 다른 픽셀 수에 따른 diff 스코어를 계산했고 rank와 suit 중 어느 쪽이든 가장 diff가 작은 템플릿 하나를 best로 저장했습니다.

- best_name : 템플릿 이름
- best_type : "Rank" or "Suit"
- best_found_angle: 몇 도로 돌렸을 때 가장 잘 맞았는지

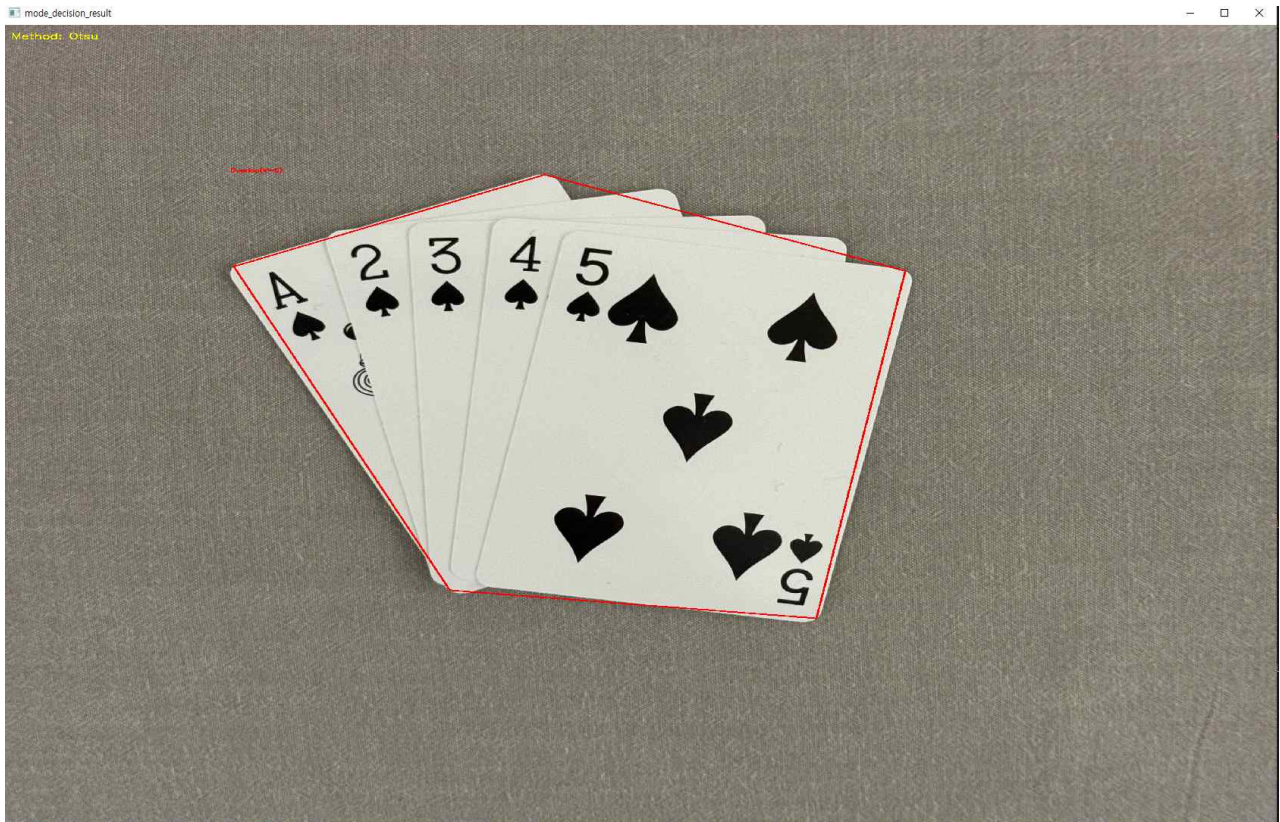
처음에 rank와 suit를 병합하지 않았기에 최종 결과를 얻기 위해서 rank와 suit를 병합하는 과정이 필요했습니다. detected_objects 리스트에서 Rank들 / Suit들을 분리하고 각 rank에 대해, 주변 일정 반경(search_radius = rank 높이 * 1.7) 안에서 아직 쓰이지 않은 suit들만 대상으로 거리를 계산했습니다. 만약 suit가 rank보다 너무 크면 제외했고 이 조건을 만족하는 suit

중 가장 가까운 것을 해당 rank와 짝으로 선택했습니다.

짝을 찾으면 `get_clean_card_info`로 "S5" 같은 카드 정보를 생성하고 `used_suit_indices`에 표시해서 같은 suit가 다른 rank와 중복 매칭되지 않게 했습니다.

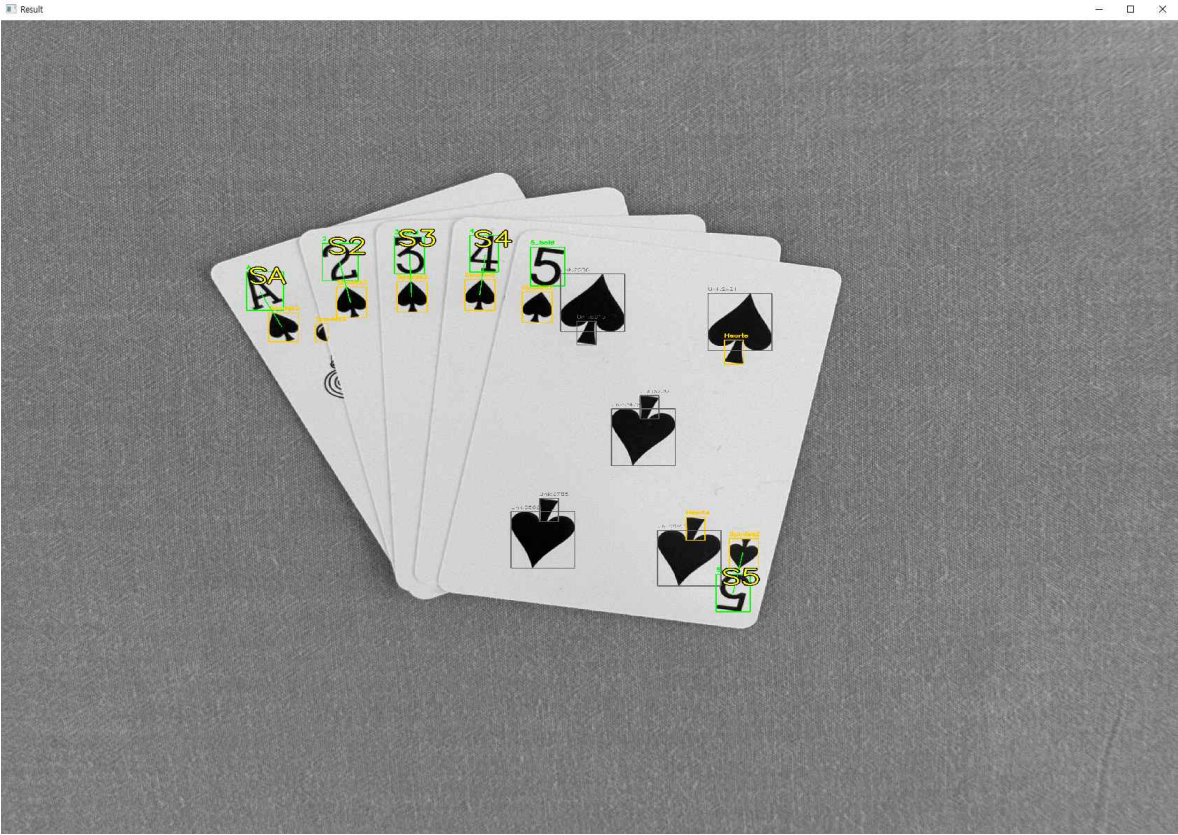
rank/suit 조합이 같은 중복 카드 제거 후 rank → suit 순으로 정렬했습니다.

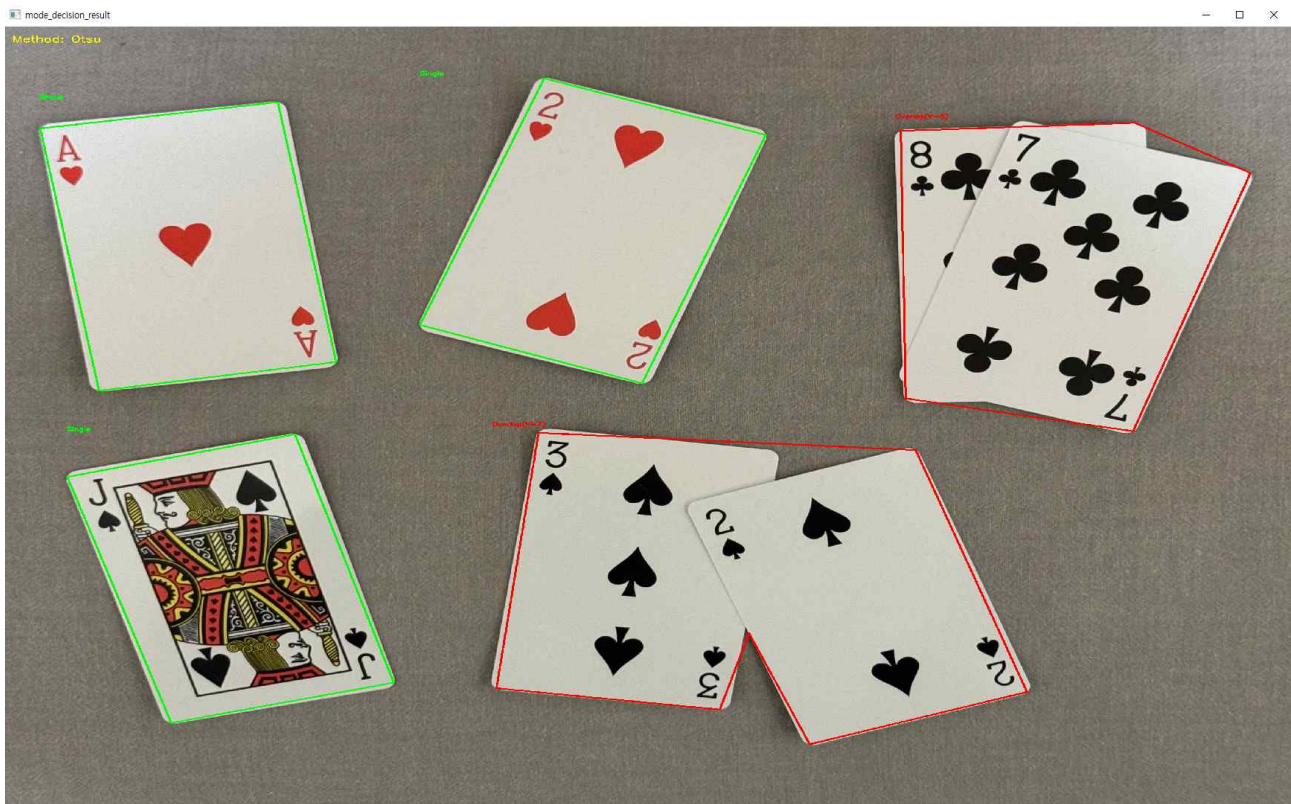
이미지에 rank와 suit 중심을 선으로 연결하고, 큰 텍스트로 문자열을 그리고 최종 카드 문자열 리스트를 반환 및 출력했습니다.



겹친 카드의 경우 Otsu로 나온 덩어리가 일반적인 경우와 다르기 때문에 겹친 로직이 실행.

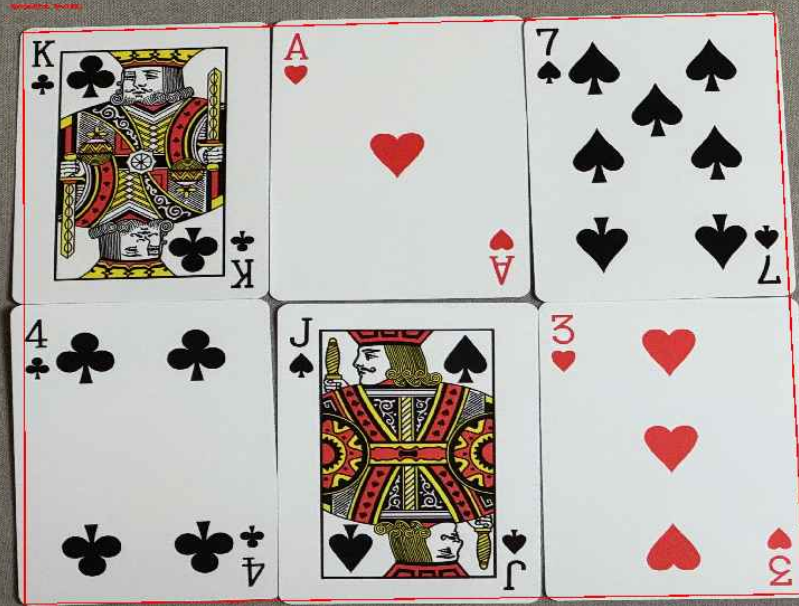
[겹친 카드를 성공적으로 분류하는 모습]



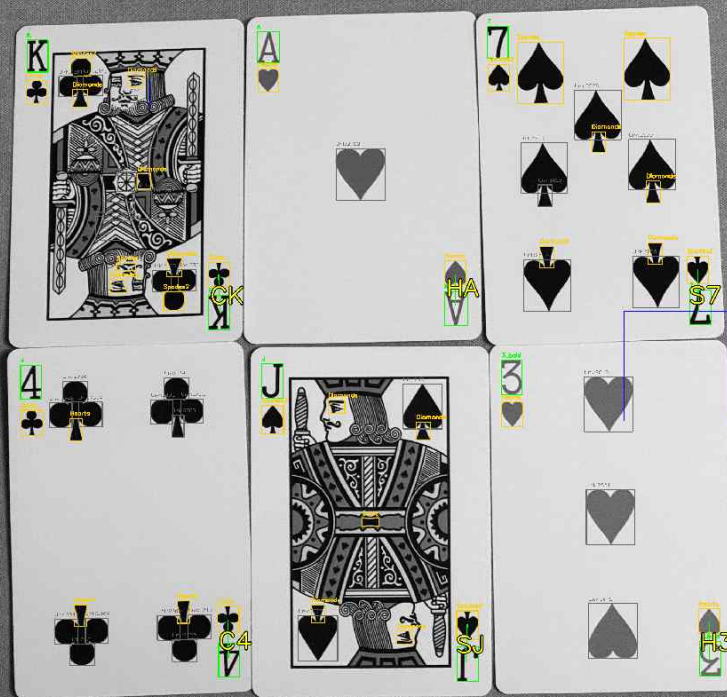


mode_decision_result

Method: CRF



Result



겹친 카드와 겹치지 않는 카드가 동시에 있는 경우에 겹친 로직이 실행돼서 정상적으로 분류하는 것을 확인할 수 있습니다.

3. 한계/오류 사례 및 개선 방안

[배경과 카드 분리]

제일 처음에 Otsu를 사용해서 카드 덩어리를 찾고 비정상적인 카드 덩어리가 탐지됐다고 판단하면 Canny 방식을 시도해야하는데 이 과정에서 만약 조건을 교묘하게 만족하지 않는 입력 이미지가 있으면 검출 및 분류가 제대로 동작하지 않습니다.

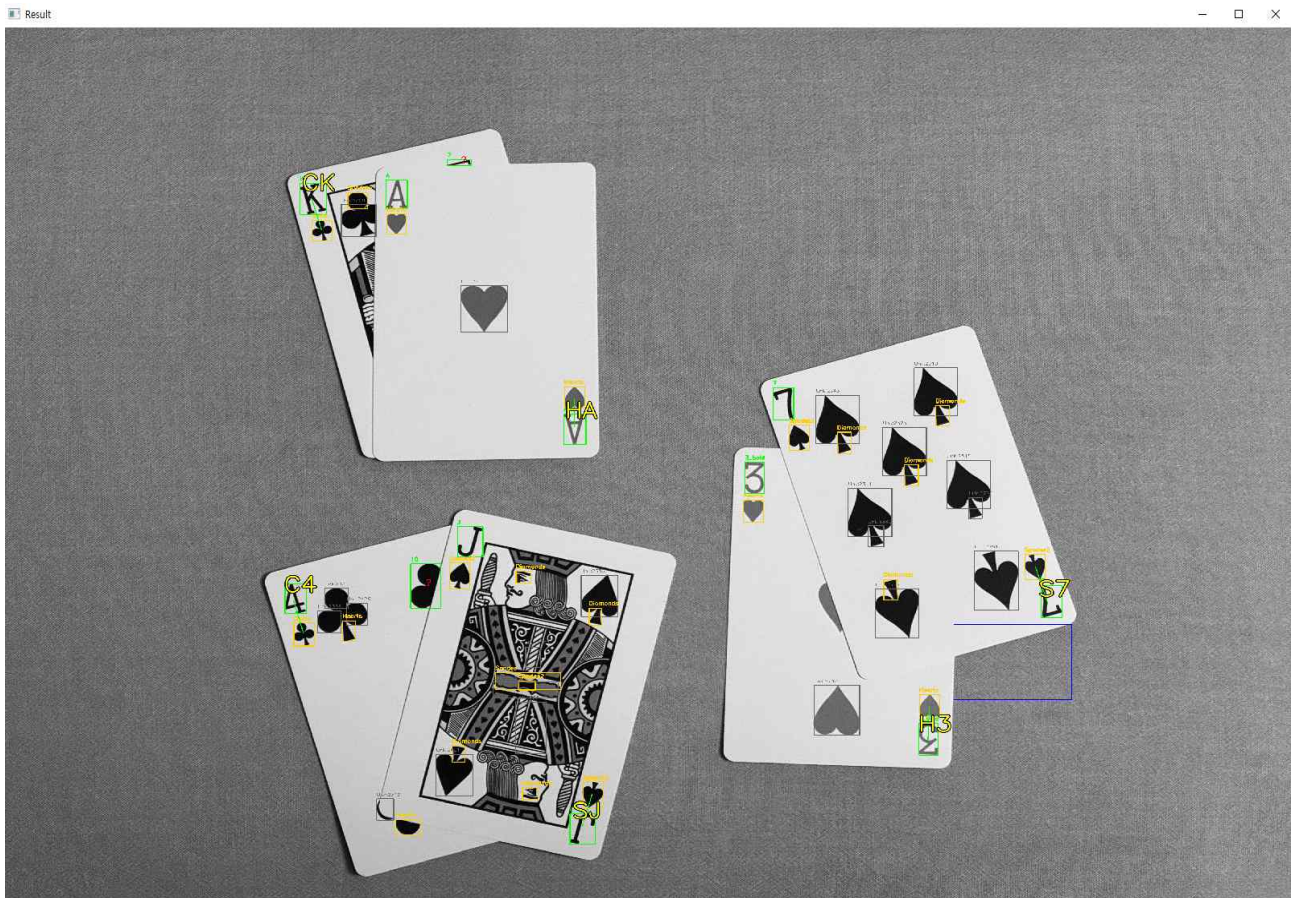
[Case 분류 로직 허점]

컨투어 분석시 단일 카드임에도 노이즈의 영향으로 꼭짓점이 4~5개 나오는 경우에도 단일 카드로 인정하는 로직이 있었습니다. 하지만 겹쳤음에도 꼭짓점을 4개로 판단하고 겹치지 않은 케이스 로직을 실행하는 문제가 발생했었습니다.



(로직 보완 전 Case 분류를 잘못된 이미지)

이런 문제를 해결하기 위해서 단일 카드의 기준을 엄격하게 꼭짓점 4개로 좁히고, 불록성 검사를 추가했습니다. 겹친 카드는 카드 사이에 움푹 들어간 부분이 생겨서 오목한 형태가 되기 때문입니다. 그 결과 케이스 분류 로직이 정상적으로 해결될 수 있었습니다.



하지만 이렇게 분류 로직을 보완했음에도 예외가 발생하는 경우가 분명 있을 것이라고 생각합니다.

만약 예외가 발생하면 Case 2를 겹친 카드가 하나도 없는 입력 이미지에서 사용되는데 이러면 제대로 분류되지 않는 문제가 있습니다.

겹친 카드가 하나도 없는 이미지에서 카드를 검출하고 분류하는건 Case1이 제일 정확하게 해주기 때문에 Case2가 개입하면 정확도가 많이 떨어지게되는 한계가 존재합니다.

이 문제는 배경과 카드 분리 과정이 완벽하게 구현됐다면 문제가 되지 않는 부분이라 배경/카드 분리를 더 정확하게 하는 것이 중요합니다.

[겹친 카드에서 rank/suit 라벨링]

라벨링이 실행할 때 카드 내부 무늬나 조명 불균형으로 인해 발생한 노이즈가 통계적 추정을 방해해서 'min_area = median_area * 3.0' 처럼 median_area 값이 이상해지고 결국 rank/suit 가 아닌 곳에 라벨링 박스가 생기는 한계가 있습니다.

다양한 입력 이미지로 테스트하면서 카드 내부에서 잡히는 라벨링 박스의 비율을 전부 다 구한 다음에 그 범위를 걸러주는 파라미터를 찾으면 해결할 수 있을 것 같습니다.

[겹친 카드의 분류]

Case2에서 라벨링으로 rank/suit을 찾고 분류하는 과정에서 결국 pairing 과정을 통해 서로 묶어줘야 합니다. 짝을 찾는 반경을 rank의 높이를 기준으로 1.7배 거리 밖의 suit는 묶지 않게 구현됐기 때문에 실제 짝이 아님에도 짝처럼 pairing 돼서 잘못 분류되는 문제가 생길 수 있습니다.

[템플릿 매칭 문제]

카메라에서 너무 멀리 떨어져있는 카드의 경우 rank보다 훨씬 작은 suit가 템플릿 매칭이 잘 되지 않는 한계가 있습니다. 예를 들어 다이아몬드이지만 멀리 떨어진걸 평탄화 시키고 템플릿 매칭을 하니 스페이드랑 헷갈리는 경우가 종종 생겼습니다.

이 문제는 템플릿 이미지를 엄청 많이 추가하거나 추가 보정을 통해서 흐릿해진걸 선명하게 만들면 개선이 가능하다고 생각합니다.

[전체적인 문제]

현재 코드는 처리속도 향상과 노이즈 제거 및 주요 에지를 강조하기 위해서 입력 이미지를 처음에 0.5배 리사이징을 시킵니다.

직접 카메라로 촬영한 이미지에 대해서는 문제가 없었지만, 인터넷에서 구한 이미지 중에서 크기가 작거나 해상도가 낮은 이미지에 대해서는 카드의 특징 사라지거나 무시되는 현상이 발생해서 검출 및 분류가 정상적으로 이뤄지지 않는 문제가 있습니다.

문제를 단기적으로 해결하기 위해 당장 리사이징 비율을 작은 입력 이미지에 맞춰서 조절하면 현재 파이프라인에서 임계값이나 비율들이 다 깨져서 직접 찍은 이미지에 대해 제대로 동작하지 않는 문제가 생기는 큰 한계가 있습니다.

이 문제는 입력 이미지의 스케일에 따라 모든 임계값을 자동 조절할 수 있게 수정하면 해결할 수 있을 것 같습니다.